



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

José Iago da Silva Lima - 500038

Juan David Bizerra Pimentel - 557340

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclométrica por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclométrica por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclométrica por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclométrica por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13
5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14

1. Caracterização do Projeto

Nome Projeto: Datachain

O **DataChain** é uma biblioteca Python de código aberto que transforma arquivos armazenados em serviços como Amazon S3, Google Cloud Storage (GCS), Azure Storage e sistemas

de arquivos locais em conjuntos de dados versionados e tipados. O projeto permite consultar e processar grandes volumes de dados não estruturados com alta performance, oferecendo recursos como processamento paralelo e distribuído, controle de versões de datasets, busca por similaridade, recuperação por checkpoints e atualizações incrementais. Além disso, integra-se a ferramentas de inteligência artificial e agentes de desenvolvimento, fornecendo uma camada de conhecimento estruturada para facilitar a análise e reutilização dos dados.

O propósito do DataChain é simplificar o gerenciamento e o processamento de dados não estruturados em larga escala, permitindo que equipes e aplicações trabalhem com arquivos distribuídos de forma eficiente e organizada. O projeto resolve problemas comuns relacionados ao armazenamento, versionamento, rastreabilidade e processamento de grandes volumes de dados, evitando a necessidade de reprocessar informações já analisadas. Com recursos como controle de linhagem dos dados, recuperação automática após falhas e integração com agentes de IA, o DataChain aumenta a produtividade, melhora a confiabilidade dos pipelines de dados e facilita a colaboração entre desenvolvedores e cientistas de dados.

2. Métricas Antes da Refatoração

Nesta seção você vai apresentar as principais métricas obtidas antes da refatoração para cada ferramenta, não precisa ser todas as métricas. Confira um exemplo abaixo:

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
src/datachain/skill/knowledge/scripts/dataset.py	3	18	52	54	F	fetch_version_data
src/datachain/skill/knowledge/scripts/dataset_all.py	4	17.75	58	71	F	_fetch_all_versions
tests/test_job_management_e2e.py	2	16	18	32	C	test_job_marked_on_exception
tests/func/test_data_storage.py	3	14.67	36	44	E	test_convert_type
src/datachain/lib/convert/python_to_sql.py	3	134.67	22	41	D	python_to_sql

Tabela de melhores métricas

arquivo	funções	cc_medi a	cc_max	cc_soma	pior_rank	pior_funcao
src/datachain/query/udf.py	3	1.0	1	3	A	__init__
examples/get_started/torch-loader.py	2	1.0	1	2	A	__init__
src/datachain/error.py	2	1.0	1	2	A	__init__
src/datachain/fs/reference.py	1	1.0	1	1	A	_open
src/datachain/cli/parser/studio.py	1	1.0	1	1	A	add_auth_parser

2.1.2 Complexidade Ciclômática por Função Antes da Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
src/datachain/skill/jobs/scripts/jobs.py	function	cmd_fetch	F	42
src/datachain/skill/knowledge/scripts/data set.py	function	fetch_version_data	F	52
src/datachain/skill/knowledge/scripts/data set_all.py	function	_fetch_all_versions	F	58
src/datachain/diff/__init__.py	function	_compare	E	31
src/datachain/lib/settings.py	method	__init__	E	31

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
noxfile.py	function	docs	A	1
noxfile.py	function	bench	A	1
noxfile.py	function	e2e	A	1
noxfile.py	function	build	A	1
noxfile.py	function	examples	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
tests/func/test_datasets.py	4.0635	C
tests/func/test_dataset_query.py	6.6484	C
src/datachain/studio.py	10.7617	B
tests/func/test_catalog.py	10.8833	B
tests/unit/test_signal_schema_partials.py	11.1101	B

Tabela de melhores métricas

arquivo	mi	rank_mi
tests/func/checkpoints/test_checkpoint_workflows.py	21.4892	A
tests/unit/test_utils.py	22.362	A
tests/func/model/test_yolo.py	22.8126	A
src/datachain/func/func.py	23.2481	A
tests/func/test_read_dataset_remote.py	23.5575	A

2.1.4 Métricas brutas Antes da Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_antes.csv*. Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
111170	52092	81555	3760	6328

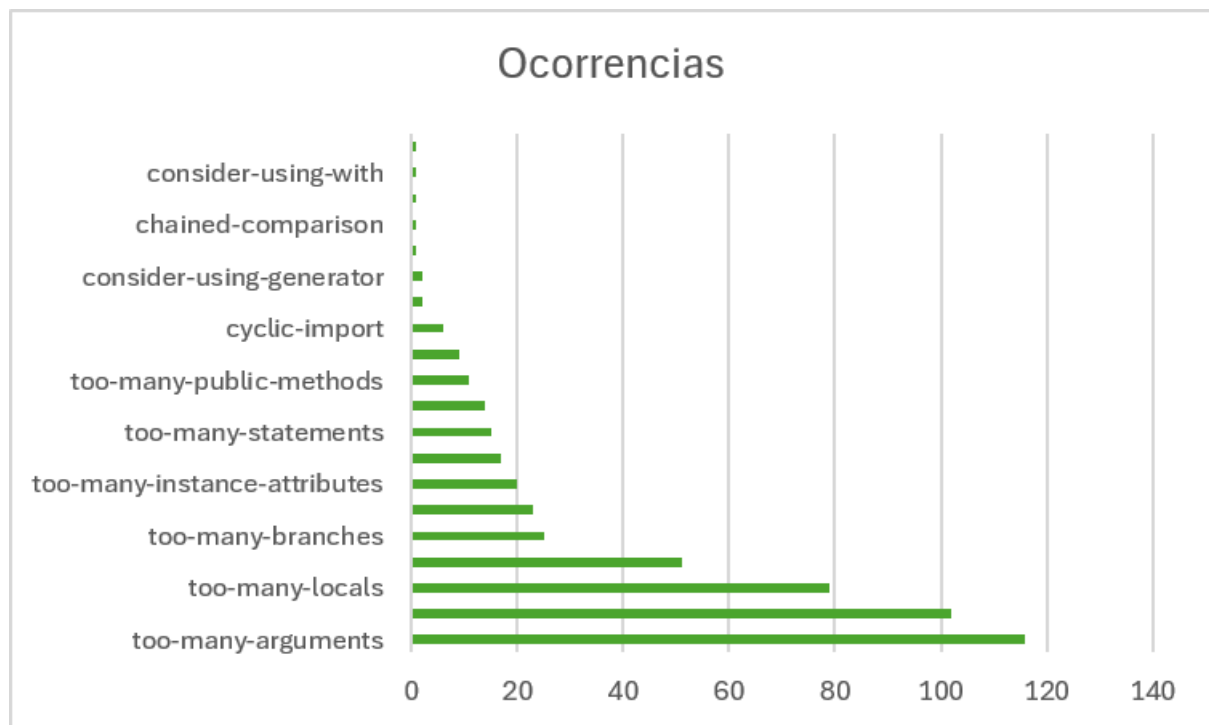
2.2 CodeCarbon Antes da Refatoração

Preencha a tabela abaixo com as métricas ambientais antes da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
5.161217500004568	1.813529405435456e-06	3.126712737913417e-07	7.991580396400002	8.616768611116439e-06	1.8439921558501e-05

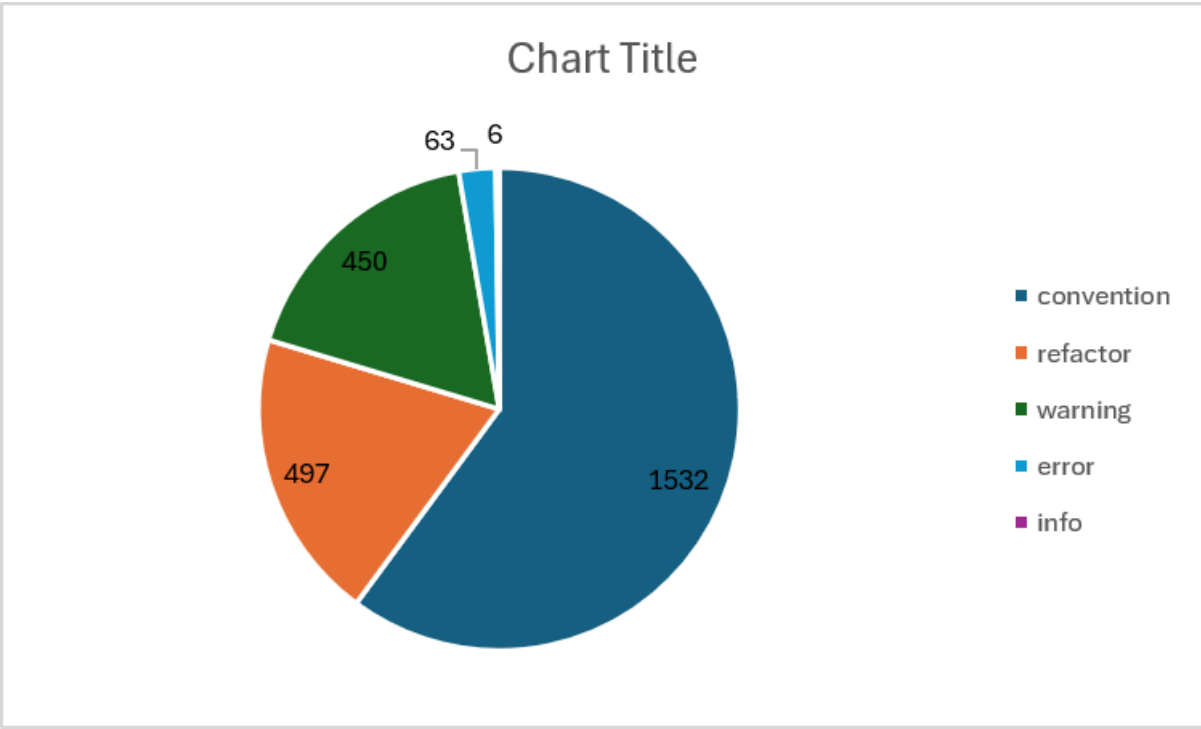
2.3 Pylint Antes da Refatoração

2.3.1 Distribuição do Smells Antes da Refatoração



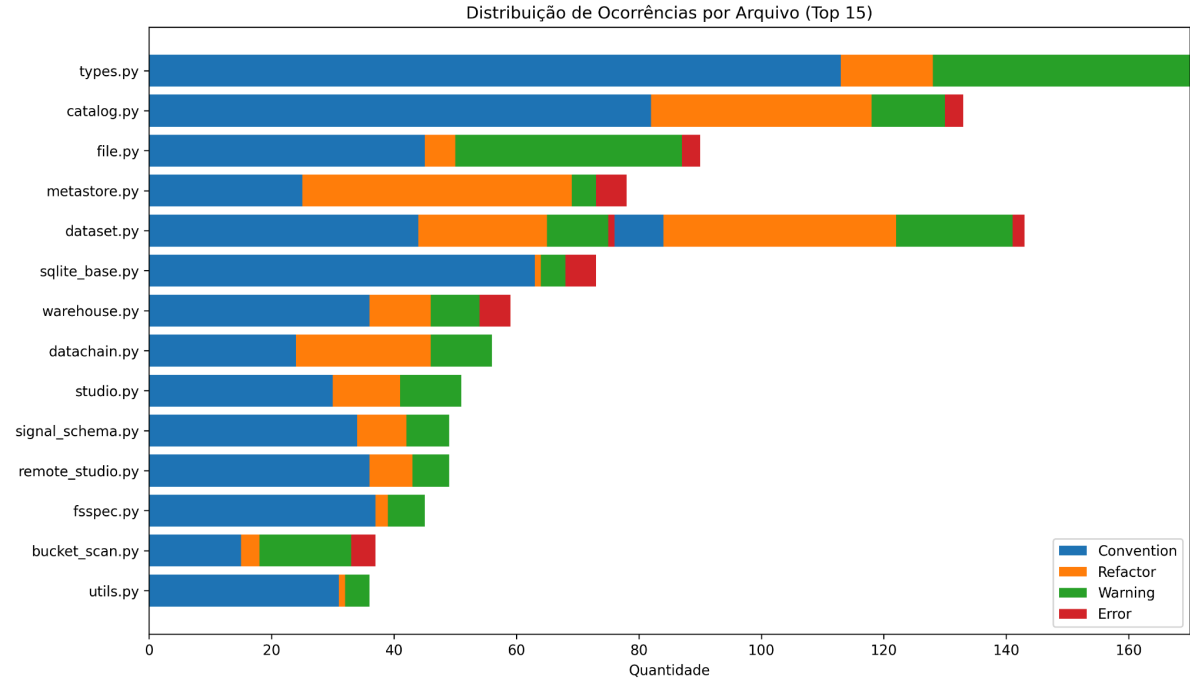
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração

A partir do JSON `pylint_distribuicao_categorias_antes.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.3 Arquivos mais Críticos Antes da Refatoração

A partir do JSON `pylint_arquivos_criticos_antes.json` gere um gráfico para representar a distribuição dos 10 arquivos mais críticos antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



2.3.4 Score Projeto Antes da Refatoração

8.62

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Gere um gráfico da cobertura de testes do projeto antes da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes antes da refatoração, execute o seguinte comando: `start "metrics-before-pytest\coverage_antes_html\index.html"`. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.

File	statements	missing	excluded	coverage
pytubefix__init__.py	18	0	0	100%
pytubefix_main_.py	545	271	0	50%
pytubefix\async_http_client.py	144	118	0	18%
pytubefix\async_youtube.py	448	389	0	13%
pytubefix\botGuard__init__.py	0	0	0	100%
pytubefix\botGuard\bot_guard.py	18	5	0	72%
pytubefix\buffer.py	19	11	0	42%
pytubefix\captions.py	101	44	0	56%
pytubefix\chapters.py	26	12	0	54%
pytubefix\cipher.py	707	675	0	5%
pytubefix\cli.py	187	45	0	76%
pytubefix\contrib__init__.py	0	0	0	100%
pytubefix\contrib\channel.py	272	143	0	47%
pytubefix\contrib\playlist.py	210	40	0	81%
pytubefix\contrib\search.py	279	195	0	30%
pytubefix\exceptions.py	187	70	0	63%
pytubefix\extract.py	227	79	0	65%
pytubefix\file_system.py	21	8	0	62%
pytubefix\helpers.py	149	26	0	83%
pytubefix\info.py	8	2	0	75%
pytubefix\innertube.py	171	88	6	49%
pytubefix\itags.py	12	0	0	100%
pytubefix\jsinterp.py	786	677	0	14%
pytubefix\keymoments.py	26	12	0	54%
pytubefix\metadata.py	32	6	0	81%
pytubefix\monostate.py	8	0	0	100%
pytubefix\parser.py	88	37	0	58%
pytubefix\protobuf.py	116	89	0	23%
pytubefix\query.py	111	9	39	92%
pytubefix\request.py	131	63	0	52%
pytubefix\sabr__init__.py	0	0	0	100%
pytubefix\sabr\common.py	97	79	0	19%
pytubefix\sabr\core__init__.py	0	0	0	100%
pytubefix\sabr\core\chunked_data_buffer.py	62	51	0	18%
pytubefix\sabr\core\server_abr_stream.py	292	201	0	31%
pytubefix\sabr\core\UMP.py	75	69	0	8%
pytubefix\sabr\proto.py	272	214	0	21%
pytubefix\sabr\video_streaming__init__.py	0	0	0	100%
pytubefix\sabr\video_streaming\buffered_range.py	163	135	0	17%
pytubefix\sabr\video_streaming\client_abr_state.py	249	240	0	4%
pytubefix\sabr\video_streaming\format_initialization_metadata.py	98	89	0	9%
pytubefix\sabr\video_streaming\media_header.py	123	112	0	9%
pytubefix\sabr\video_streaming\next_request_policy.py	51	43	0	16%
pytubefix\sabr\video_streaming\playback_cookie.py	43	34	0	21%
pytubefix\sabr\video_streaming\sabr_error.py	32	25	0	22%
pytubefix\sabr\video_streaming\sabr_redirect.py	26	19	0	27%
pytubefix\sabr\video_streaming\stream_protection_status.py	39	27	0	31%
pytubefix\sabr\video_streaming\streamer_context.py	448	395	0	12%
pytubefix\sabr\video_streaming\time_range.py	44	36	0	18%
pytubefix\sabr\video_streaming\video_playback_abr_request.py	291	254	0	13%
pytubefix\sigsig__init__.py	0	0	0	100%
pytubefix\sigsig\node_runner.py	81	58	0	28%
pytubefix\streams.py	261	66	0	75%
pytubefix\version.py	3	1	0	67%
Total	7797	5262	45	33%

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram antes da refatoração. Para isso, basta visualizar a página HTML `pytest_antes.html`. Para acessá-la, execute o seguinte comando: `start "metrics-before-pytest\pytest_antes.html"`.

Quantidade de testes que passaram: 168

Quantidade de testes que falharam: 39

3. Processo de Refatoração

Modelo utilizado: DeepSeek V4 Flash Free

3.1 too-many-statements

Quantidade de ocorrências: 15

module	obj	message
src.datachain.studio	show_logs_from_client	Too many statements (64/50)
src.datachain.catalog.catalog	Catalog.pull_dataset	Too many statements (98/50)
src.datachain.cli.commands.skill	install_skills	Too many statements (53/50)
src.datachain.cli.parser	get_parser	Too many statements (114/50)
src.datachain.diff	_compare	Too many statements (57/50)
src.datachain.lib.settings	Settings.__init__	Too many statements (69/50)
src.datachain.lib.signal_schema	SignalSchema.to_partial	Too many statements (69/50)
src.datachain.lib.dc.datachain	DataChain.group_by	Too many statements (82/50)
src.datachain.lib.dc.storage	read_storage	Too many statements (56/50)
src.datachain.query.dataset	UDFStep.populate_udf_output_table	Too many statements (69/50)
src.datachain.skill.jobs.scripts.jobs	cmd_fetch	Too many statements (63/50)
src.datachain.skill.knowledge.scripts.dataset	fetch_version_data	Too many statements (108/50)
src.datachain.skill.knowledge.scripts.dataset_all	_fetch_all_versions	Too many statements (128/50)
src.datachain.skill.knowledge.scripts.plan	plan_datasets	Too many statements (53/50)
src.datachain.sql.sqlite.base	setup	Too many statements (55/50)

3.1.1 Ocorrência Número 1

Tabela de identificação

Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração: Sim, o plano de refatoração era muito bom

Link da publicação OpenCode: <https://opncd.ai/share/lp7hzFm9>

Print consumo de token:

```
Plano de resolução too-many-  
statements studio.py
```

Context

```
68.193 tokens
```

```
34% used
```

```
$0.00 spent
```

LSP

```
LSPs are disabled
```

Análise da refatoração: Valeu a pena usar a LLM como se tratava de algo muito simples foi uma boa solução

3.1.2 Ocorrência Número 2

Seguiu o plano de refatoração: Não, ele sugeriu fazer muitas mudanças e no final ainda tava errado

link do opencode: <https://opncd.ai/share/HxnY1sVi>

print do consumo de token:

```
Too-many-statements em catalog.py
```

Context

```
17.878 tokens
```

```
9% used
```

```
$0.00 spent
```

LSP

```
LSPs are disabled
```

3.1.3 Ocorrência Número 3

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/4OI2gNvZ>

Foto:

```
Plano resolução too-many-statements
em skill.py
https://opnacd.ai/share/40I2gNvZ

Context
61.714 tokens
31% used
$0.00 spent

LSP
LSPs are disabled
```

3.1.4 Ocorrência Número 4

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opnacd.ai/share/hBDk5nGi>

Foto:

```
Plano resolução too-many-statements
parser

Context
62.520 tokens
31% used
$0.00 spent

LSP
LSPs are disabled
```

3.1.5 Ocorrência Número 5

Seguiu o plano de refatoração: Sim, o plano de refatoração era muito bom

link: <https://opnacd.ai/share/sSHooSdr>

foto:

```
Plano para too-many-statements no
__init__.py
https://opnacd.ai/share/sSHooSdr

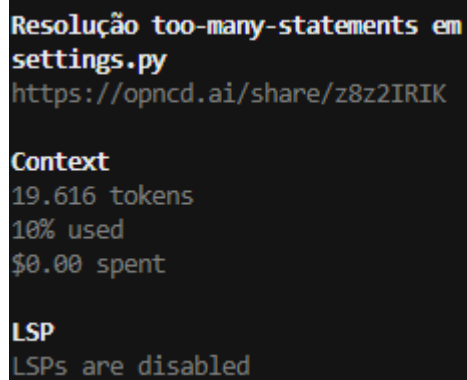
Context
41.544 tokens
21% used
$0.00 spent

LSP
LSPs are disabled
```

3.1.6 Ocorrência Número 6

Seguiu o plano de refatoração: Sim, o plano de refatoração era muito bom

link:<https://opncd.ai/share/z8z2IRIK>



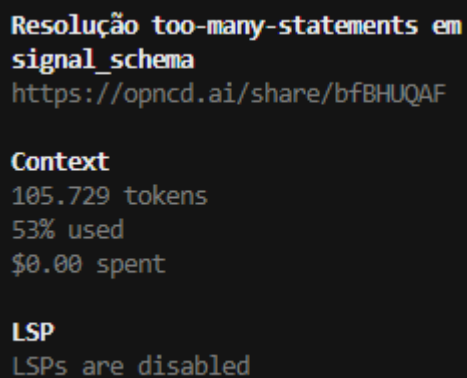
```
Resolução too-many-statements em  
settings.py  
https://opncd.ai/share/z8z2IRIK  
  
Context  
19.616 tokens  
10% used  
$0.00 spent  
  
LSP  
LSPs are disabled
```

foto:

3.1.7 Ocorrência Número 7

Seguiu o plano de refatoração: Sim, o plano de refatoração era muito bom

link:<https://opncd.ai/share/bfBHUQAF>



```
Resolução too-many-statements em  
signal_schema  
https://opncd.ai/share/bfBHUQAF  
  
Context  
105.729 tokens  
53% used  
$0.00 spent  
  
LSP  
LSPs are disabled
```

foto:

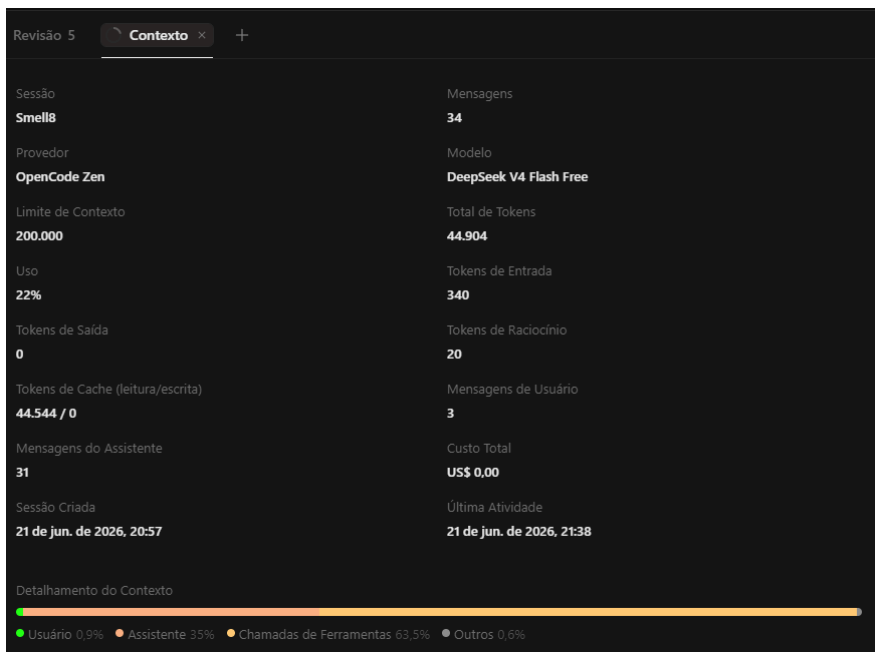
3.1.8 Ocorrência Número 8

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/TNXGzqUp>

Análise:

foto:



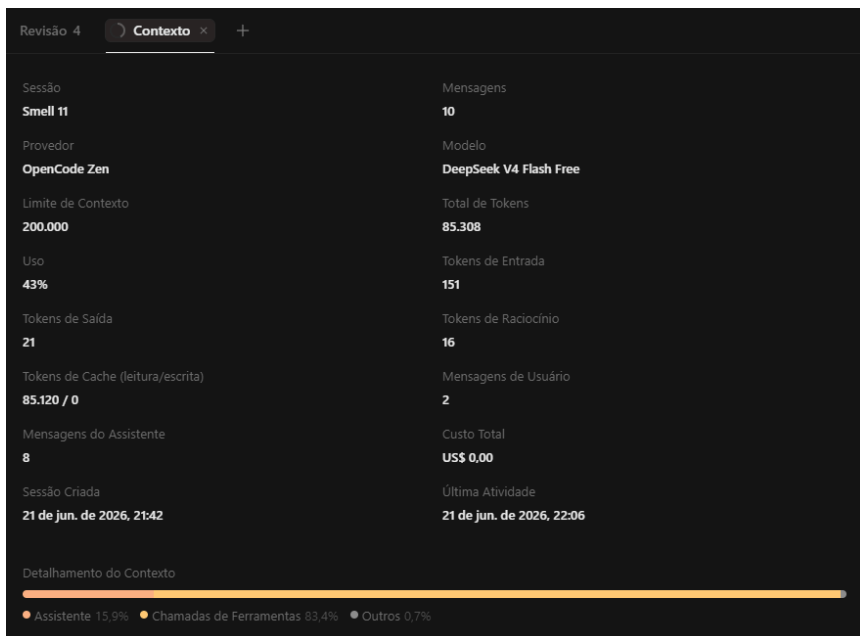
3.1.9 Ocorrência Número 9

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/wpXJhslG>

Análise:

foto:



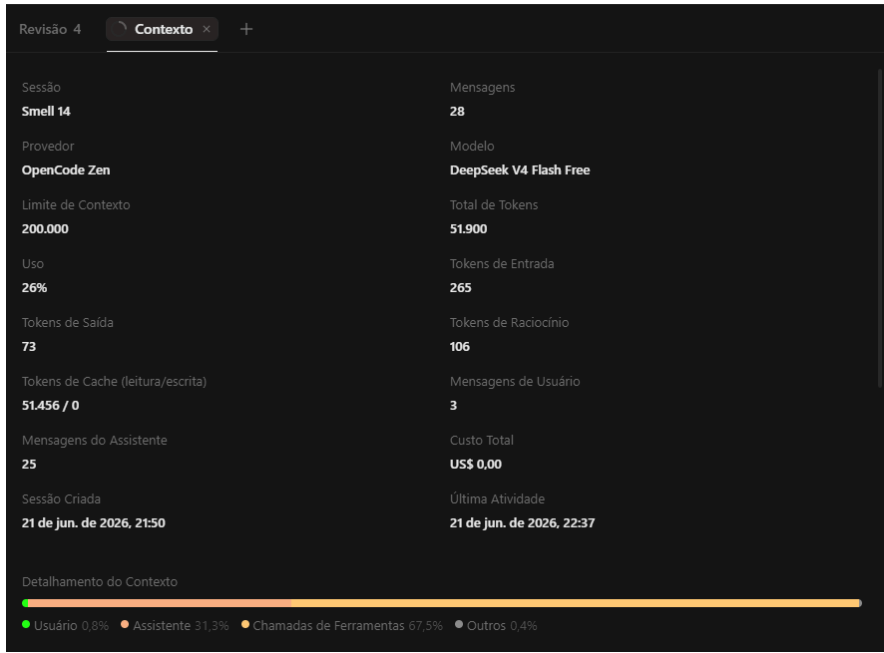
3.1.10 Ocorrência Número 10

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/0InOxk0m>

Análise:

foto:



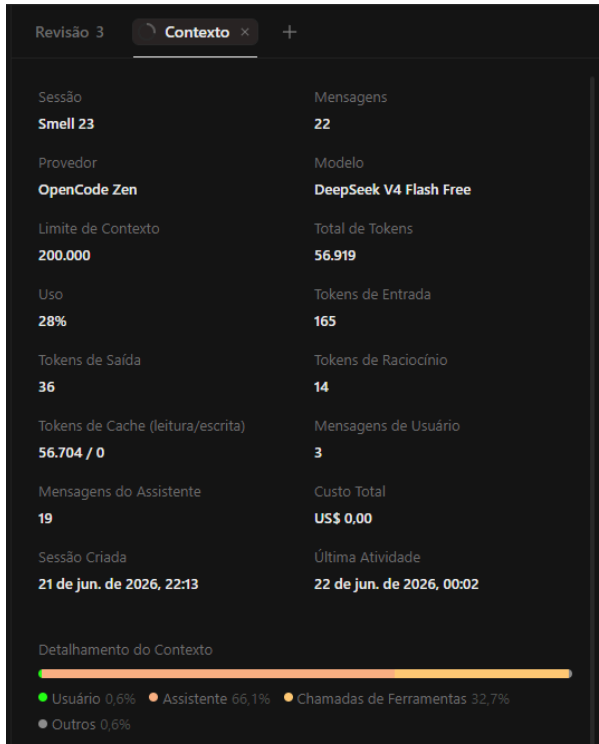
3.1.11 Ocorrência Número 11

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/qGljohBN>

Análise:

foto:



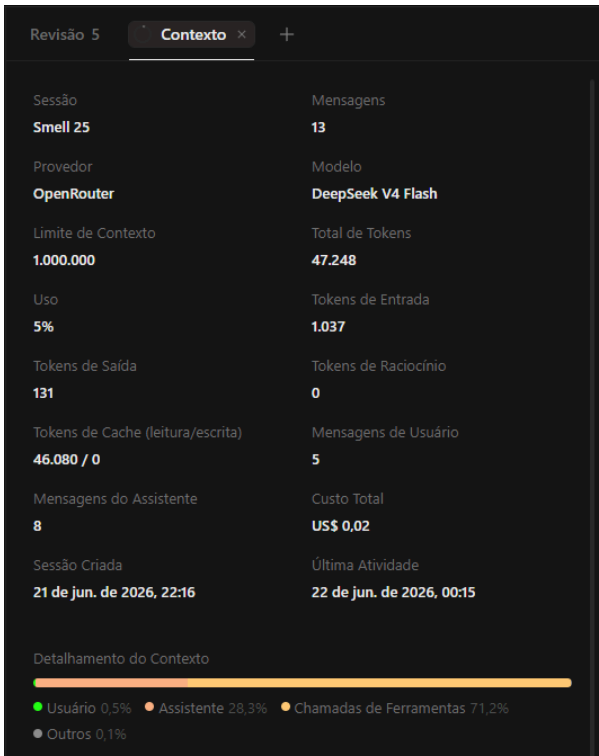
3.1.12 Ocorrência Número 12

Seguiu o plano de refatoração: Sim, o plano era bom e fácil de fazer

link: <https://opncd.ai/share/LqIVseou>

Análise:

foto:



3.1.13 Ocorrência Número 13

link: <https://opncd.ai/share/3sCISQWp>

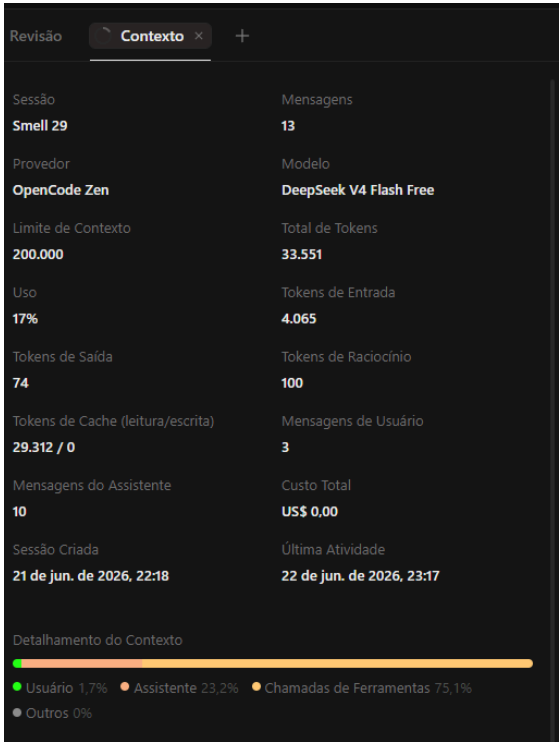
foto:



3.1.14 Ocorrência Número 14

link: <https://opncd.ai/share/9NOiW4vI>

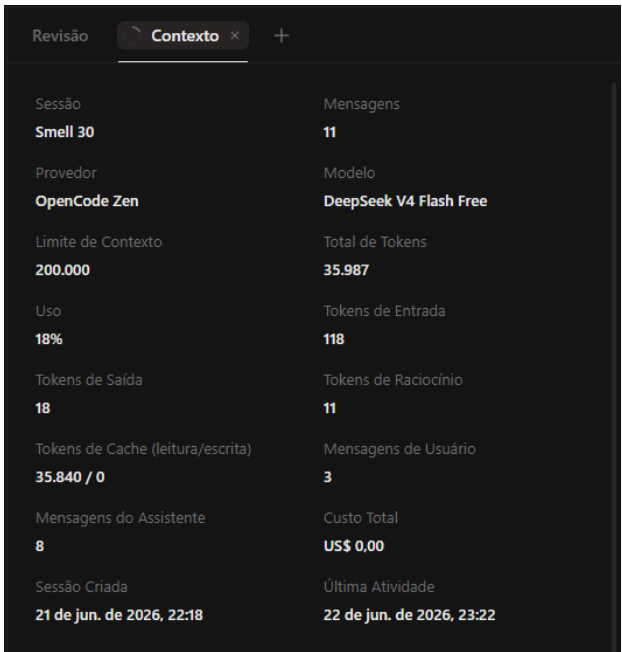
foto:



3.1.15 Ocorrência Número 15

link: <https://opncd.ai/share/LDCaS1i7>

foto:



3.2 too-many-instance-attributes

Quantidade de ocorrências: 20

module	obj	message
src.datachain.asyn	src.datachain.asyn	Too many instance attributes (10/7)
src.datachain.checkpoint_event	src.datachain.checkpoint_event	Too many instance attributes (19/7)
src.datachain.dataset	src.datachain.dataset	Too many instance attributes (8/7)
src.datachain.dataset	src.datachain.dataset	Too many instance attributes (20/7)
src.datachain.dataset	src.datachain.dataset	Too many instance attributes (13/7)
src.datachain.dataset	src.datachain.dataset	Too many instance attributes (18/7)
src.datachain.dataset	src.datachain.dataset	Too many instance attributes (8/7)
src.datachain.job	src.datachain.job	Too many instance attributes (17/7)
src.datachain.node	src.datachain.node	Too many instance attributes (10/7)
src.datachain.catalog.catalog	src.datachain.catalog.catalog	Too many instance attributes (10/7)
src.datachain.catalog.dependency	src.datachain.catalog.dependency	Too many instance attributes (11/7)
src.datachain.func.func	src.datachain.func.func	Too many instance attributes (12/7)
src.datachain.lib.pytorch	src.datachain.lib.pytorch	Too many instance attributes (15/7)
src.datachain.lib.settings	src.datachain.lib.settings	Too many instance attributes (9/7)
src.datachain.query.dataset	src.datachain.query.dataset	Too many instance attributes (10/7)
src.datachain.query.dataset	src.datachain.query.dataset	Too many instance attributes (9/7)
src.datachain.query.dataset	src.datachain.query.dataset	Too many instance attributes (10/7)
src.datachain.query.dataset	src.datachain.query.dataset	Too many instance attributes (21/7)
src.datachain.query.dispatch	src.datachain.query.dispatch	Too many instance attributes (18/7)
src.datachain.query.dispatch	src.datachain.query.dispatch	Too many instance attributes (13/7)

3.2.1 Ocorrência Número 1

Seguiu o plano de refatoração:

Link da publicação OpenCode:<https://opncd.ai/share/mNikHHKc>

```
Refatorar too-many-instance-  
attributes no asyn.py  
https://opncd.ai/share/mNikHHKc
```

```
Context  
38.754 tokens  
19% used  
$0.00 spent
```

```
LSP  
LSPs are disabled
```

Print consumo de token:

Análise da refatoração:

3.2.2 Ocorrência Número 2

Seguiu o plano de refatoração:

link do opencode:<https://opncd.ai/share/8NMZolko>

```
Too-many-instance-attributes em  
checkpoint_event
```

```
Context  
70.627 tokens  
35% used  
$0.00 spent
```

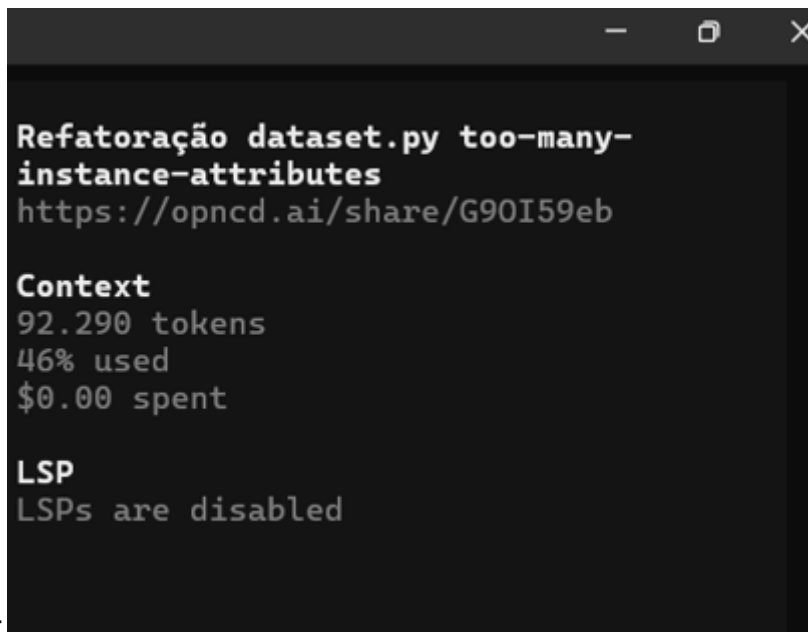
```
LSP  
LSPs are disabled
```

print do consumo de token:

3.2.3 Ocorrência Número 3

Seguiu o plano de refatoração:

link:<https://opncd.ai/share/G9OI59eb>



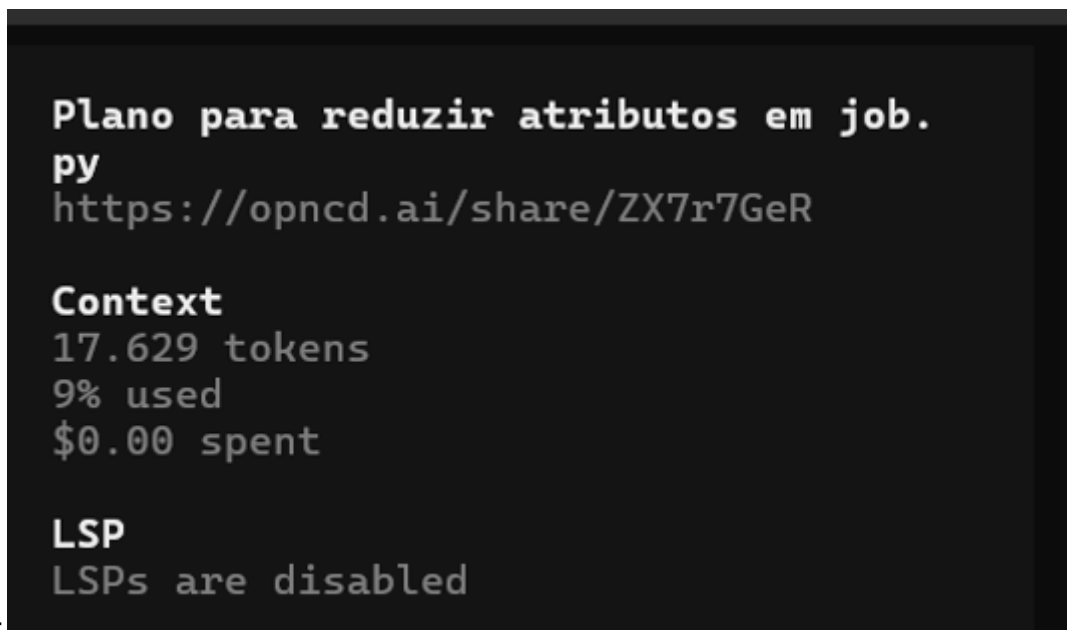
```
Refatoração dataset.py too-many-  
instance-attributes  
https://opnacd.ai/share/G90I59eb  
  
Context  
92.290 tokens  
46% used  
$0.00 spent  
  
LSP  
LSPs are disabled
```

Foto:

3.2.4 Ocorrência Número 4

Seguiu o plano de refatoração:

link: <https://opnacd.ai/share/ZX7r7GeR>



```
Plano para reduzir atributos em job.  
py  
https://opnacd.ai/share/ZX7r7GeR  
  
Context  
17.629 tokens  
9% used  
$0.00 spent  
  
LSP  
LSPs are disabled
```

Foto:

3.2.5 Ocorrência Número 5

link: <https://opnacd.ai/share/Ing9MvFL>

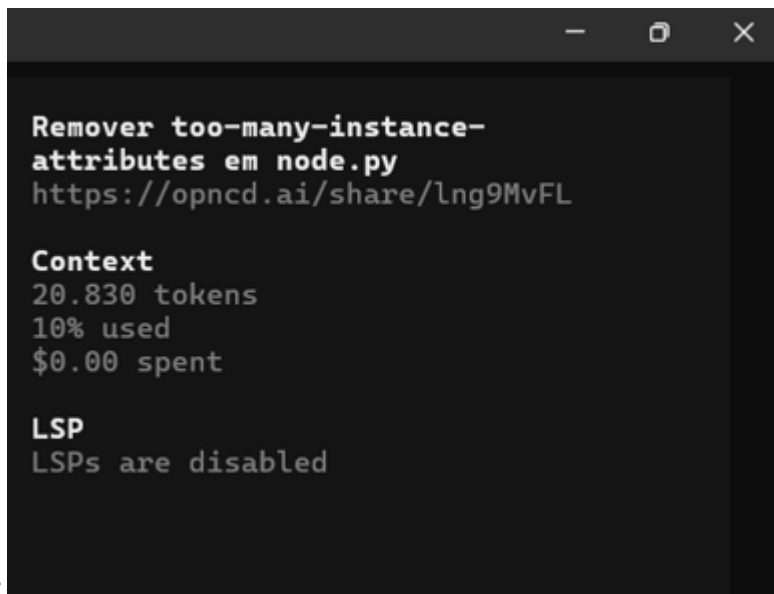


foto:

3.2.6 Ocorrência Número 6

link:<https://opnacd.ai/share/V2i6duDw>

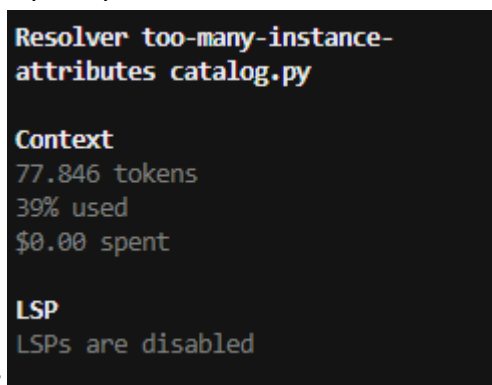


foto:

3.2.7 Ocorrência Número 7

link:

foto:

3.2.8 Ocorrência Número 8

link:

foto:

3.2.9 Ocorrência Número 9

link:

foto:

3.2.10 Ocorrência Número 10

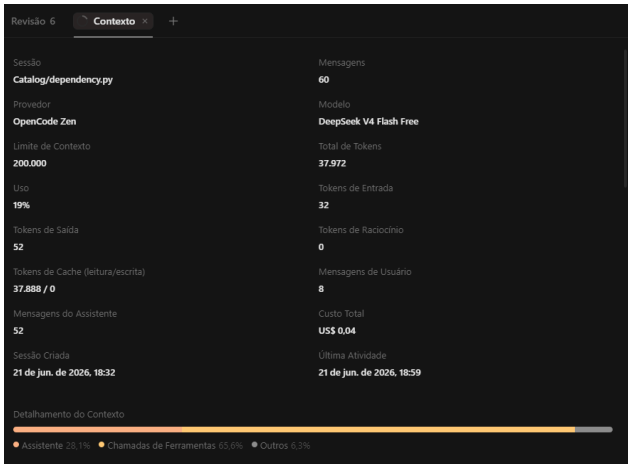
link:

foto:

3.2.11 Ocorrência Número 11

link:<https://opncd.ai/share/Cj6Ewhyf>

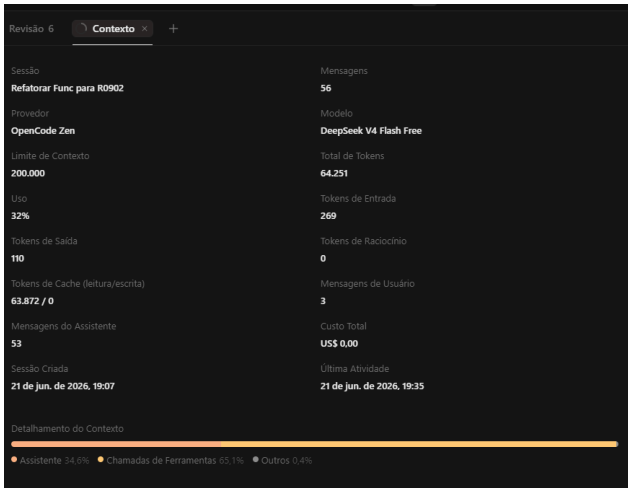
foto:



3.2.12 Ocorrência Número 12

link: <https://opncd.ai/share/vdz1P0En>

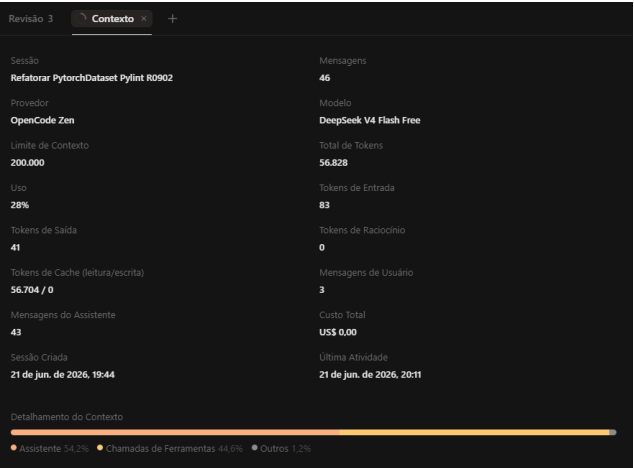
foto:



3.2.13 Ocorrência Número 13

link:<https://opncd.ai/share/u9QCb67f>

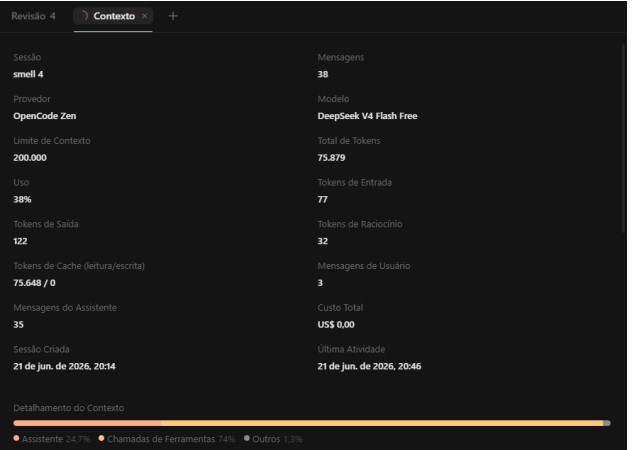
foto:



3.2.14 Ocorrência Número 14

link: <https://opnecd.ai/share/qBZEzPP8>

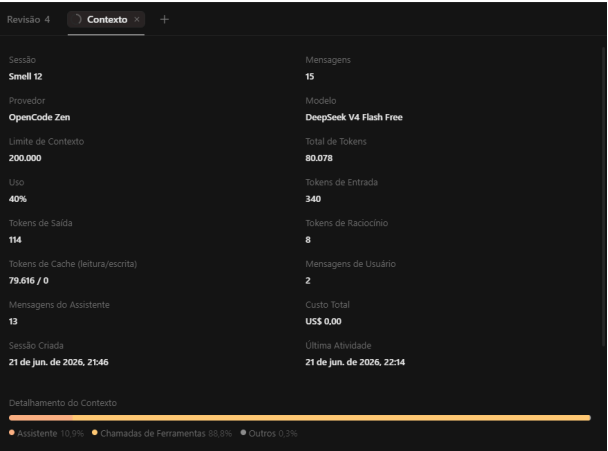
foto:



3.2.15 Ocorrência Número 15

link: <https://opnecd.ai/share/PKshaCXG>

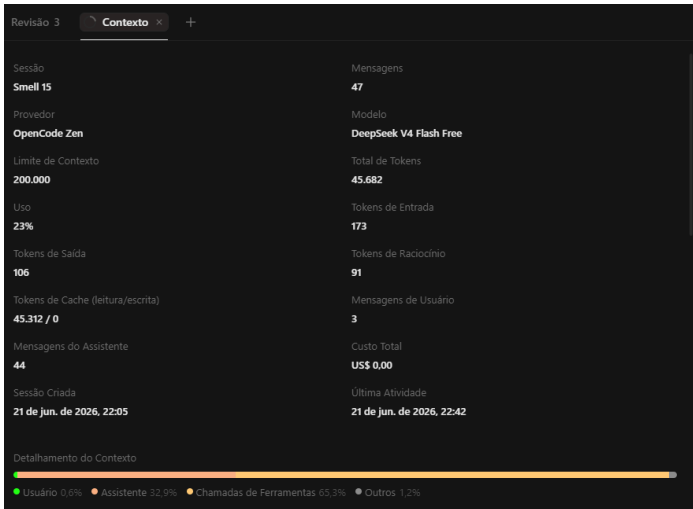
foto:



3.2.16 Ocorrência Número 16

link: <https://opncd.ai/share/QfzTQ5K4>

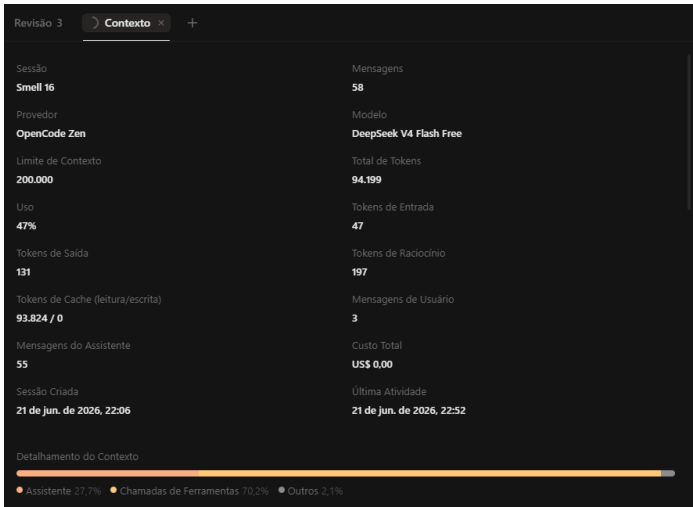
foto:



3.2.17 Ocorrência Número 17

link: <https://opncd.ai/share/E0F7Tiik>

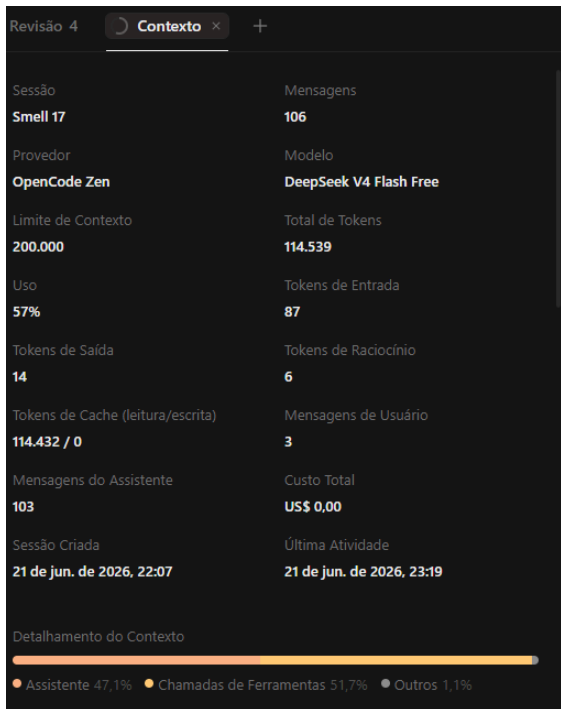
foto:



3.2.18 Ocorrência Número 18

link: <https://opncd.ai/share/aJ4VMys6>

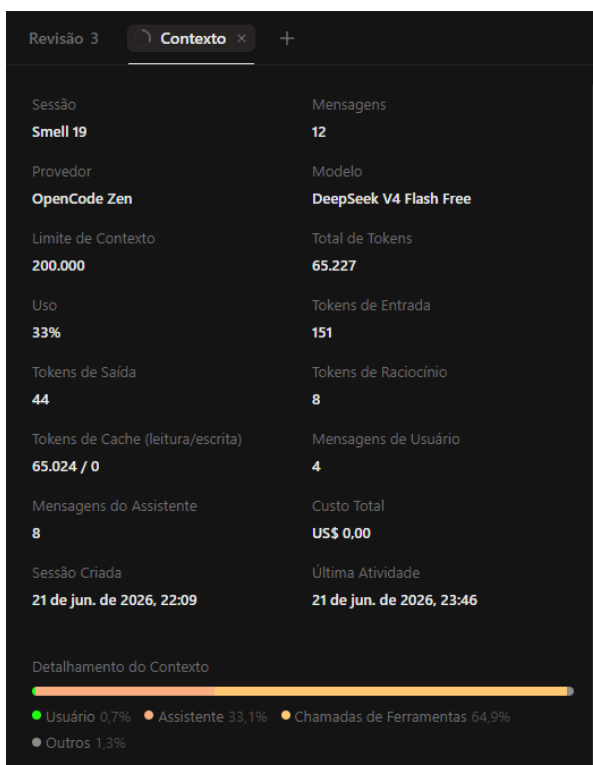
foto:



3.2.19 Ocorrência Número 19

link: <https://opncd.ai/share/EkgBSqCM>

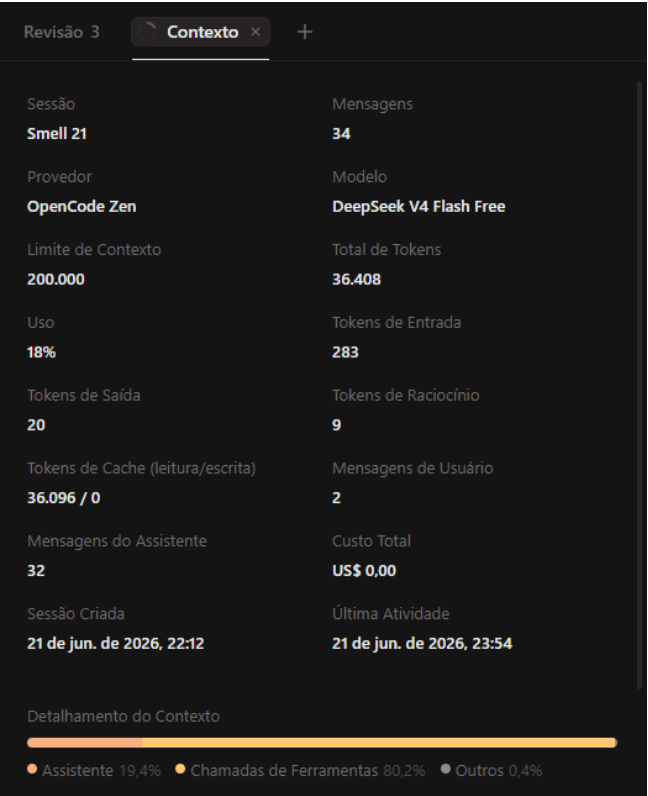
foto:



3.2.20 Ocorrência Número 20

link: <https://opncd.ai/share/tKnyOZgq>

foto:



3.3 too-many-branches

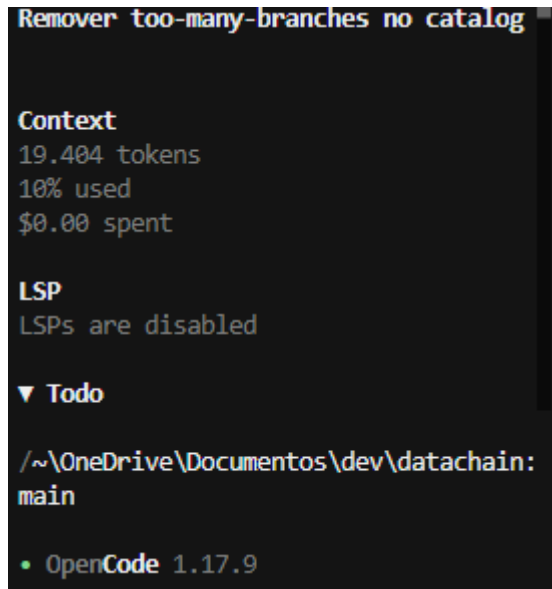
Quantidade de ocorrências: 25

module	obj	message
src.datachain.catalog.catalog	prepare_output_for_cp	Too many branches (15/12)
src.datachain.catalog.catalog	Catalog.enlist_sources_grouped	Too many branches (13/12)
src.datachain.catalog.catalog	Catalog.pull_dataset	Too many branches (20/12)
src.datachain.cli.commands.skill	install_skills	Too many branches (15/12)
src.datachain.data_storage.metastore	AbstractDBMetastore.update_dataset	Too many branches (14/12)
src.datachain.data_storage.metastore	AbstractDBMetastore.update_dataset_version	Too many branches (13/12)
src.datachain.diff	_compare	Too many branches (16/12)
src.datachain.lib.arrow	arrow_type_mapper	Too many branches (14/12)
src.datachain.lib.clip	clip_similarity_scores	Too many branches (14/12)
src.datachain.lib.settings	Settings.__init__	Too many branches (34/12)
src.datachain.lib.signal_schema	SignalSchema._resolve_type	Too many branches (13/12)
src.datachain.lib.signal_schema	SignalSchema._convert_feature_value	Too many branches (15/12)
src.datachain.lib.udf_signature	UdfSignature.parse	Too many branches (15/12)
src.datachain.lib.utils	type_to_str	Too many branches (22/12)
src.datachain.lib.convert.python_to_sql	python_to_sql	Too many branches (14/12)
src.datachain.lib.dc.datachain	DataChain.group_by	Too many branches (28/12)
src.datachain.lib.dc.datachain	DataChain.subtract	Too many branches (15/12)
src.datachain.lib.dc.datasets	read_dataset	Too many branches (13/12)
src.datachain.query.dataset	UDFStep.populate_udf_output_table	Too many branches (15/12)
src.datachain.query.dataset	DatasetQuery.save	Too many branches (14/12)
src.datachain.query.dispatch	UDFDispatcher.run_udf_parallel	Too many branches (20/12)
src.datachain.skill.jobs.scripts.jobs	cmd_fetch	Too many branches (16/12)
src.datachain.skill.knowledge.scripts.dataset	fetch_version_data	Too many branches (30/12)
src.datachain.skill.knowledge.scripts.dataset_all	_fetch_all_versions	Too many branches (28/12)
src.datachain.skill.knowledge.scripts.plan	plan_datasets	Too many branches (15/12)

3.3.1 Ocorrência Número 01

Link: <https://opncd.ai/share/YnbD6VH3>

Foto:



3.3.2 Ocorrência Número 2

Link: <https://opncd.ai/share/Moc3aBpB>

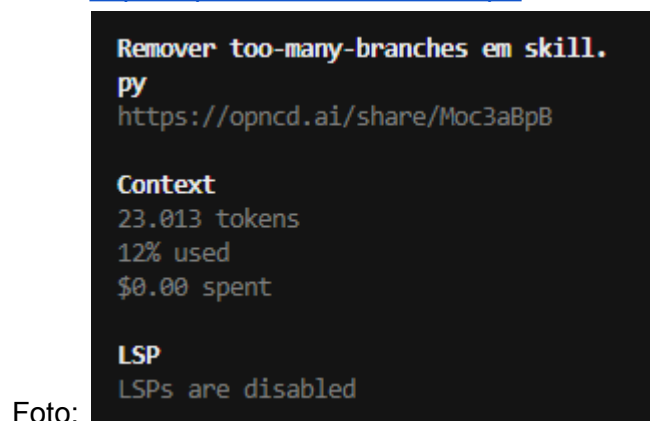


Foto:

3.3.3 Ocorrência Número 3

link: <https://opncd.ai/share/72962fz1>

Foto:

**Plano remover too-many-branches em
metastore.py**

<https://opnecd.ai/share/72962fz1>

Context

67.457 tokens

34% used

\$0.00 spent

LSP

LSPs are disabled

3.3.4 Ocorrência Número 4

link: <https://opnecd.ai/share/72962fz1>

Foto:

**Plano remoção too-many-branches
arrow.py**

Context

8.528 tokens

4% used

\$0.00 spent

LSP

LSPs are disabled

3.3.5 Ocorrência Número 5

Link: <https://opnecd.ai/share/72962fz1>

Foto:

**Remoção de too-many-branches em
diff/__init__.py**

Context

17.396 tokens

9% used

\$0.00 spent

LSP

LSPs are disabled

3.3.6 Ocorrência Número 6

link:<https://opnacd.ai/share/eEbb6YEZ>

**Plano remoção too-many-branches
arrow.py**

Context

8.528 tokens

4% used

\$0.00 spent

LSP

LSPs are disabled

foto:

3.3.7 Ocorrência Número 7

link:<https://opnacd.ai/share/9kQg6m8b>

Plano para remover too-many-branches em clip.py
<https://opnacd.ai/share/9kQg6m8b>

Context

16.363 tokens

8% used

\$0.00 spent

LSP

LSPs are disabled

foto:

3.3.8 Ocorrência Número 8

link:<https://opnacd.ai/share/lUcPpdAq>

Reduzir too-many-branches em settings.py a 10
<https://opnacd.ai/share/lUcPpdAq>

Context

33.470 tokens

17% used

\$0.00 spent

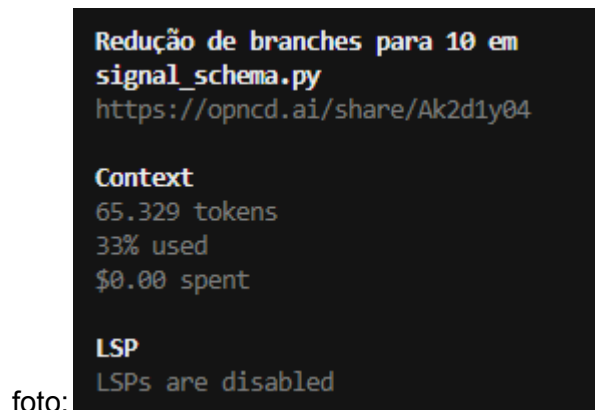
LSP

LSPs are disabled

foto:

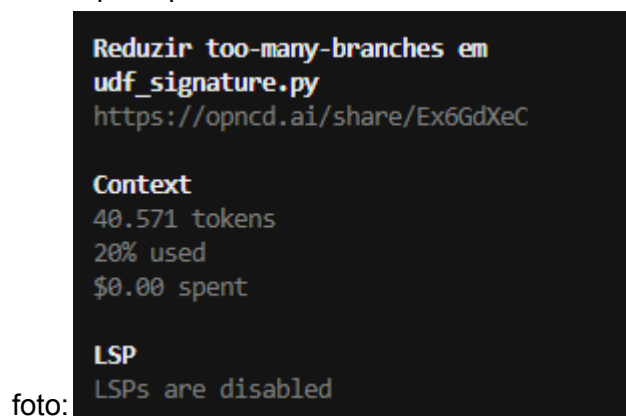
3.3.9 Ocorrência Número 9

link:<https://opnacd.ai/share/Ak2d1y04>



3.3.10 Ocorrência Número 10

link:<https://opnacd.ai/share/Ex6GdXeC>



3.3.11 Ocorrência Número 11

link:

foto:

3.3.12 Ocorrência Número 12

link:

foto:

3.3.13 Ocorrência Número 13

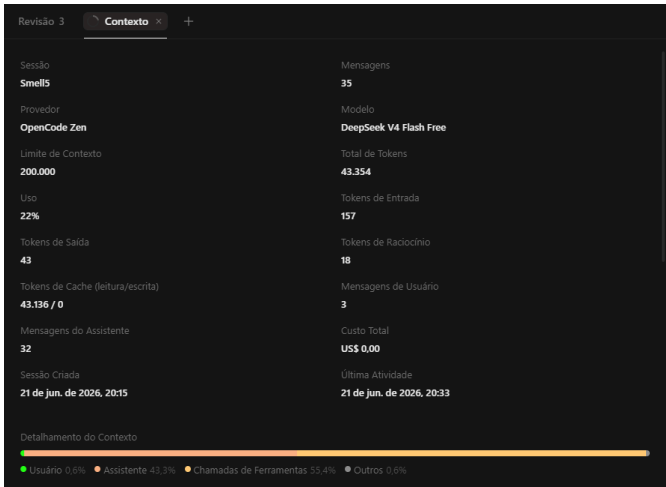
link:

foto:

3.3.14 Ocorrência Número 14

link:<https://opncd.ai/share/288EJVtW>

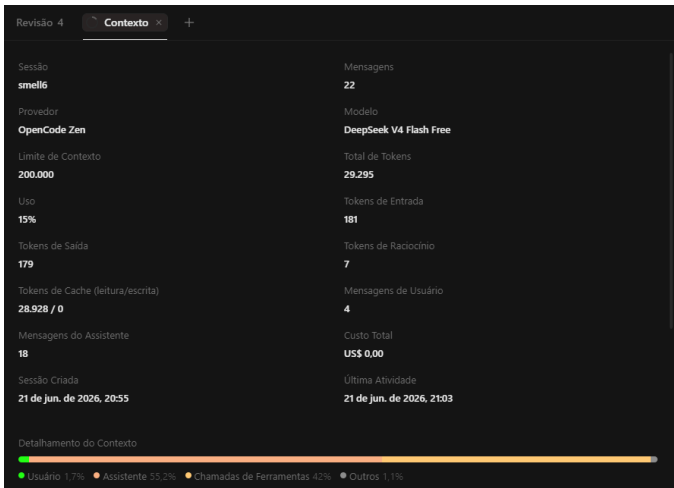
foto:



3.3.15 Ocorrência Número 15

link:<https://opncd.ai/share/uZ37T9yd>

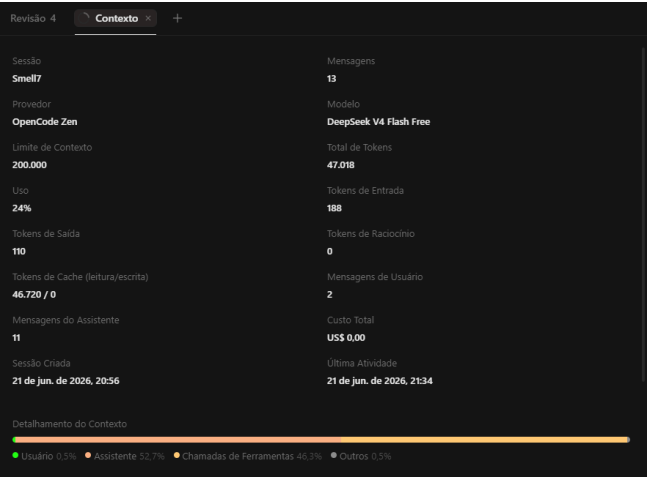
foto:



3.3.16 Ocorrência Número 16

link:<https://opncd.ai/share/4yeqr9NG>

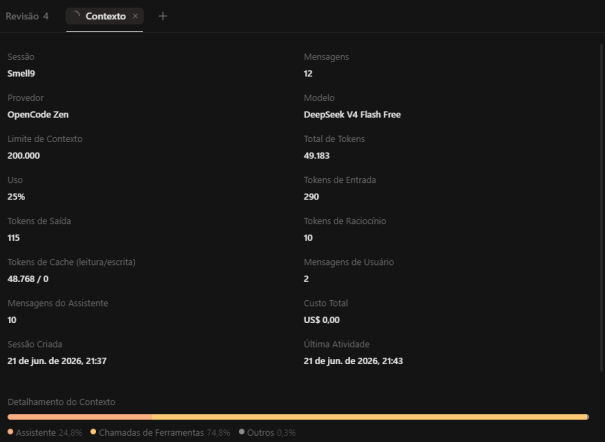
foto:



3.3.17 Ocorrência Número 17

link:<https://opncd.ai/share/yCcXNyTR>

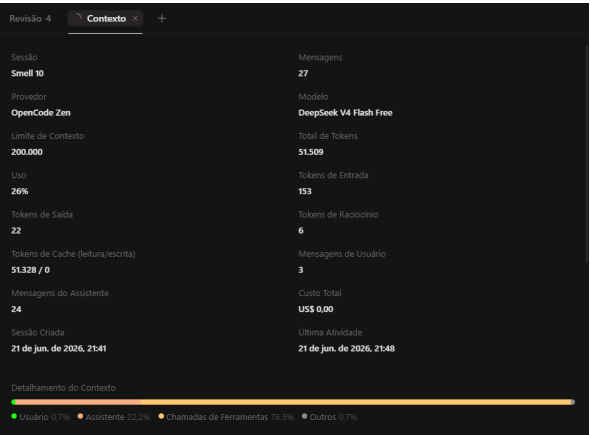
foto:



3.3.18 Ocorrência Número 18

link:<https://opncd.ai/share/OFwsdzR8>

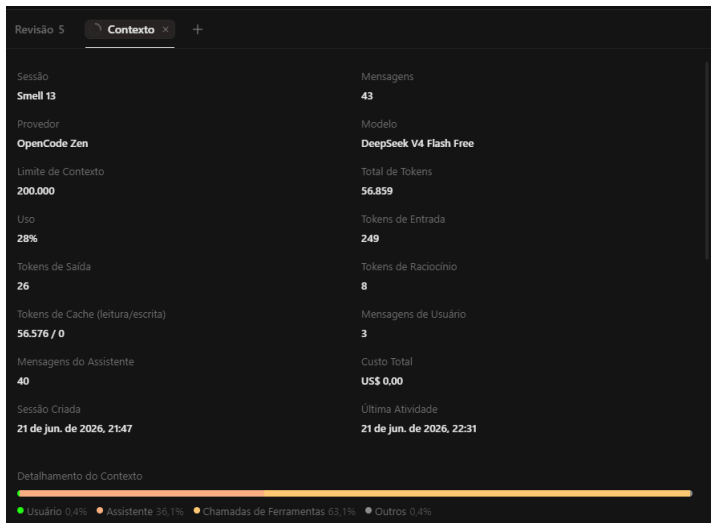
foto:



3.3.19 Ocorrência Número 19

link:<https://opncd.ai/share/MyTp2gOU>

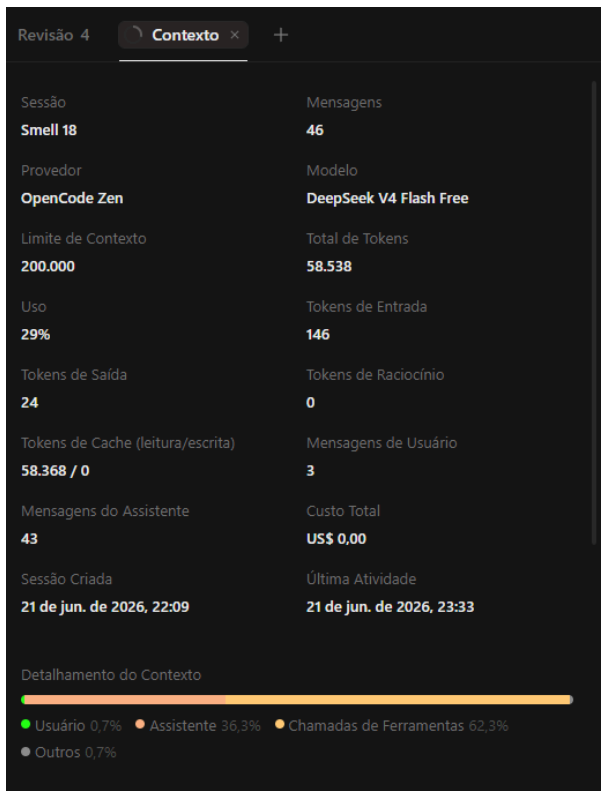
foto:



3.3.20 Ocorrência Número 20

link:<https://opncd.ai/share/10ys2yg8>

foto:



3.3.21 Ocorrência Número 21

link:<https://opncd.ai/share/dTMkoQWz>

foto:

Revisão 3 Contexto x +

Sessão	Mensagens
Smell 20	26
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	39.540
Uso	Tokens de Entrada
20%	281
Tokens de Saída	Tokens de Raciocínio
335	12
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
38.912 / 0	3
Mensagens do Assistente	Custo Total
23	US\$ 0,00
Sessão Criada	Última Atividade
21 de jun. de 2026, 22:12	21 de jun. de 2026, 23:50

Detalhamento do Contexto

Usuário 1,1% Assistente 50,2% Chamadas de Ferramentas 48,4% Outros 0,4%

3.3.22 Ocorrência Número 22

link:<https://opncd.ai/share/OKOcx7dz>

foto:

Revisão 3 Contexto x +

Sessão	Mensagens
Smell 22	41
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	54.681
Uso	Tokens de Entrada
27%	111
Tokens de Saída	Tokens de Raciocínio
22	20
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
54.528 / 0	3
Mensagens do Assistente	Custo Total
38	US\$ 0,00
Sessão Criada	Última Atividade
21 de jun. de 2026, 22:13	21 de jun. de 2026, 23:58

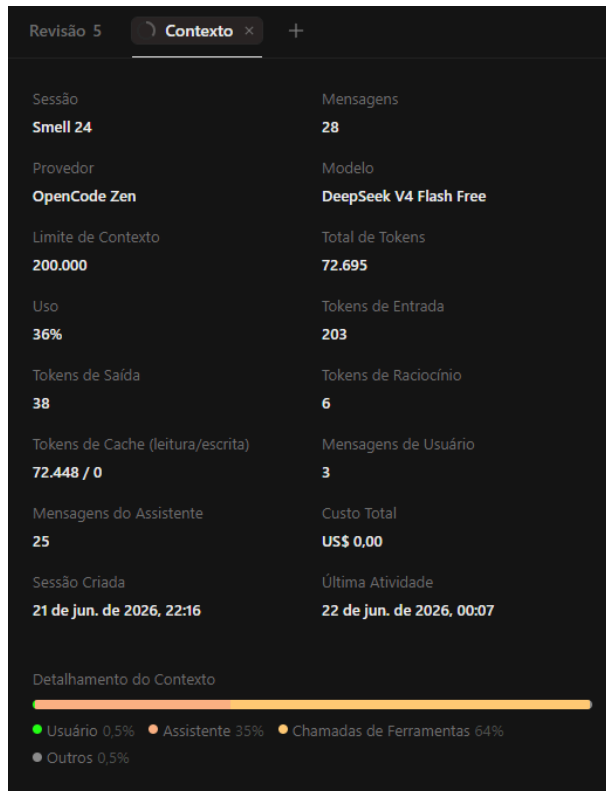
Detalhamento do Contexto

Usuário 0,9% Assistente 29,7% Chamadas de Ferramentas 68,5% Outros 0,9%

3.3.23 Ocorrência Número 23

link: <https://opnecd.ai/share/km1Lfl5B>

foto:



3.3.24 Ocorrência Número 24

link: <https://opnecd.ai/share/rbCTqFyU>

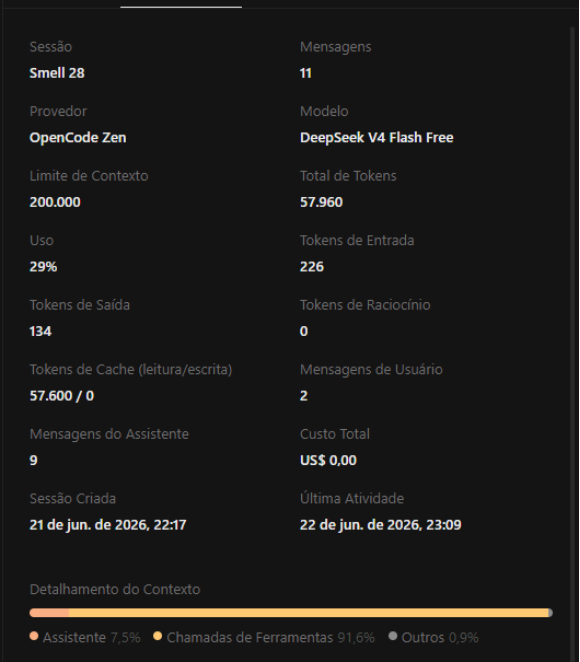
foto:



3.3.25 Ocorrência Número 25

link:<https://opncd.ai/share/3ApKH0dI>

foto:



4. Métricas Após a Refatoração

Para as métricas após a refatoração você deve adicionar os mesmos tipos de informações que foram apresentadas anteriormente. Atenção, certifique-se de que cada um dos dados estão nas pastas *metrics-after-...*

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclômática por Arquivo Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
build/lib/datachain/skill/jobs/scripts/jobs.py	11	7	42	77	F	cmd_fetch
build/lib/datachain/skill/knowledge/scripts/dataset_all.py	4	17.75	58	71	F	_fetch_all_versions
build/lib/datachain/skill/knowledge/scripts/dataset.py	3	18	52	54	F	fetch_version_data
tests/unit/lib/test_datachain.py	299	3.7	32	1105	E	test_column_compute
src/datachain/lib/settings.py	13	5.38	21	70	E	__init__

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
build/lib/datachain/checkpoint_event.py	1	1	1	1	A	parse
build/lib/datachain/error.py	2	1	1	2	A	__init__
build/lib/datachain/cli/commands/index.py	1	1	1	1	A	index
build/lib/datachain/cli/parser/job.py	1	1	1	1	A	add_jobs_parser
build/lib/datachain/cli/parser/pipeline.py	1	1	1	1	A	add_pipeline_parser

4.1.2 Complexidade Ciclômática por Função Após a Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
build/lib/datachain/skill/jobs/scripts/jobs.py	function	cmd_fetch	F	42
build/lib/datachain/skill/knowledge/scripts/dataset.py	function	fetch_version_data	F	52
build/lib/datachain/skill/knowledge/scripts/dataset_all.py	function	_fetch_all_versions	F	58
src/datachain/lib/settings.py	method	__init__	E	31
tests/unit/lib/test_datachain.py	function	test_column_compute	E	32

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
extract_metrics_before_radon.py	function	normalizar_caminho	A	1
extract_metrics_before_radon.py	function	rodar_radon	A	1
noxfile.py	function	docs	A	1
noxfile.py	function	bench	A	1
noxfile.py	function	e2e	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
build/lib/datachain/catalog/catalog.py	0	C
build/lib/datachain/data_storage/metastore.py	0	C
build/lib/datachain/lib/signal_schema.py	0	C
build/lib/datachain/lib/dc/datachain.py	0	C
build/lib/datachain/query/dataset.py	0	C

Tabela de melhores métricas

arquivo	mi	rank_mi
---------	----	---------

build/lib/datachain/checkpoint.py	100.0	A
build/lib/datachain/error.py	100.0	A
build/lib/datachain/nodes_fetcher.py	100.0	A
build/lib/datachain/plugins.py	100.0	A
build/lib/datachain/telemetry.py	100.0	A

4.1.4 Métricas Brutas Após a Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_depois.csv*. Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
164063	77811	118156	5325	12452

4.2 CodeCarbon Após a Refatoração

Preencha a tabela abaixo com as métricas ambientais depois da refatoração.

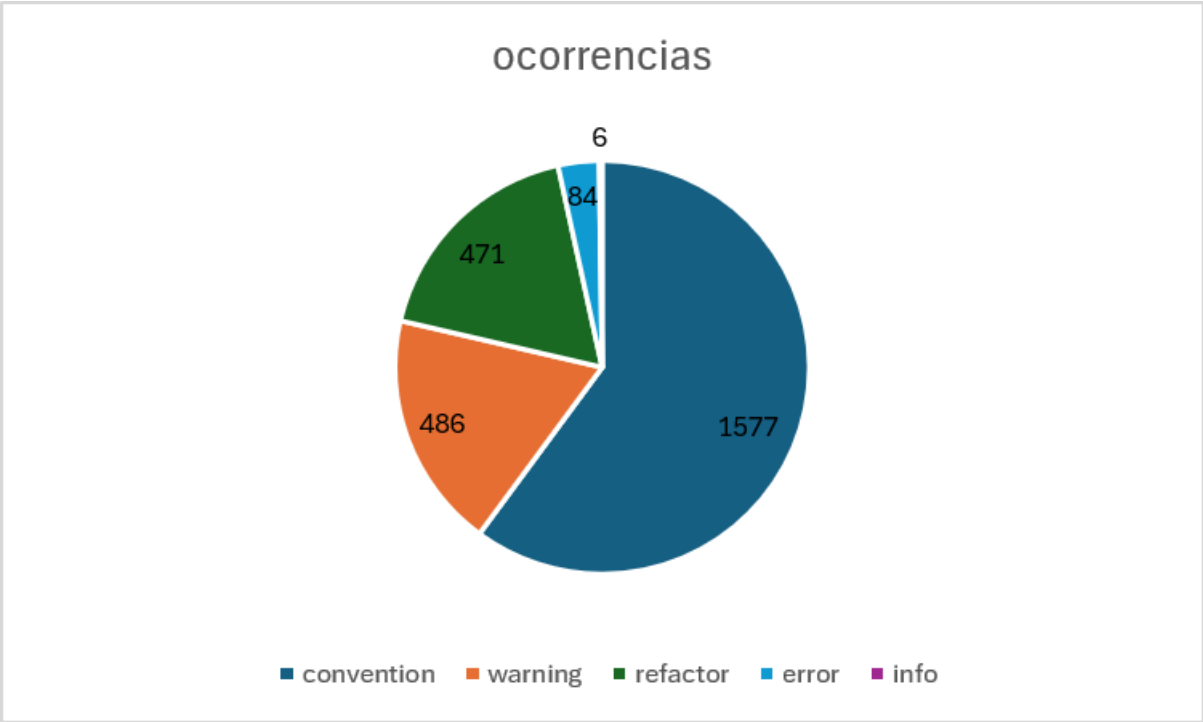
duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
7.28549 7399978 34	1.4375287402415832 e-06	1.9731374006747393 e-07	2.3929838176	10.0	1.4616756215089102e- 05

4.3 Pylint Após a Refatoração

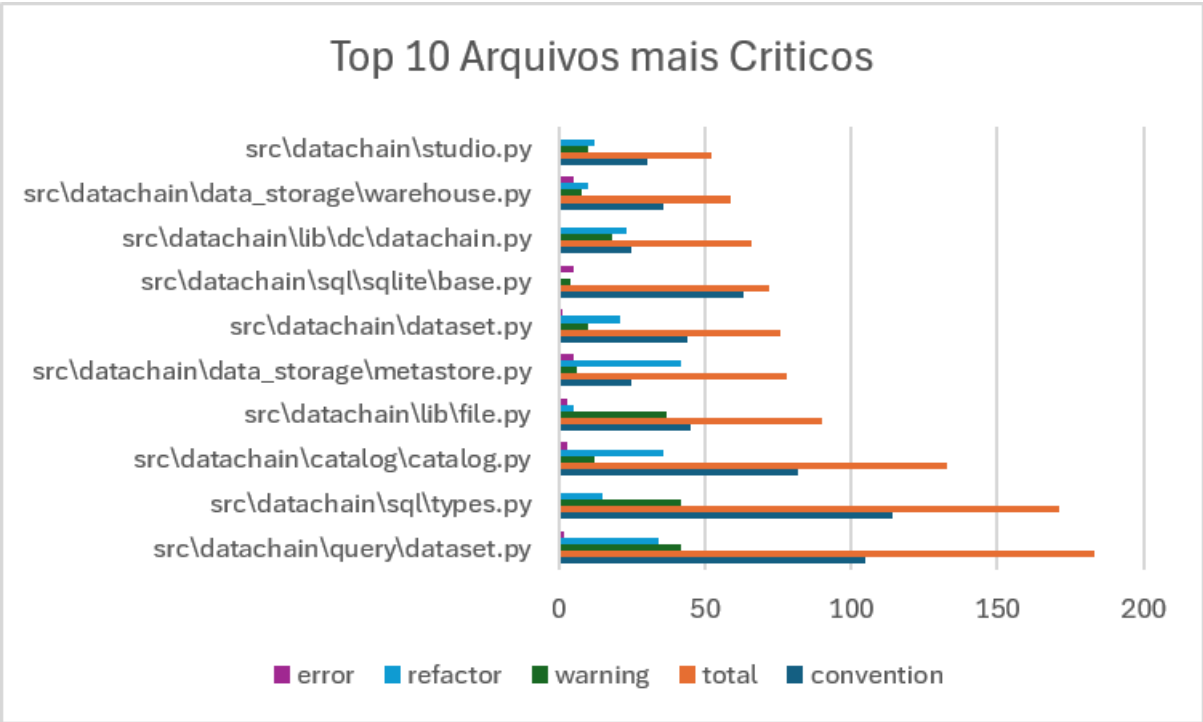
4.3.1 Distribuição do Smells Após a Refatoração

A partir do JSON *pylint_ranking_smells_depois.json* gere um gráfico para representar a distribuição dos code smells depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.

4.3.2 Distribuição dos Problemas de Código Após a Refatoração



4.3.3 Arquivos mais Críticos Após a Refatoração

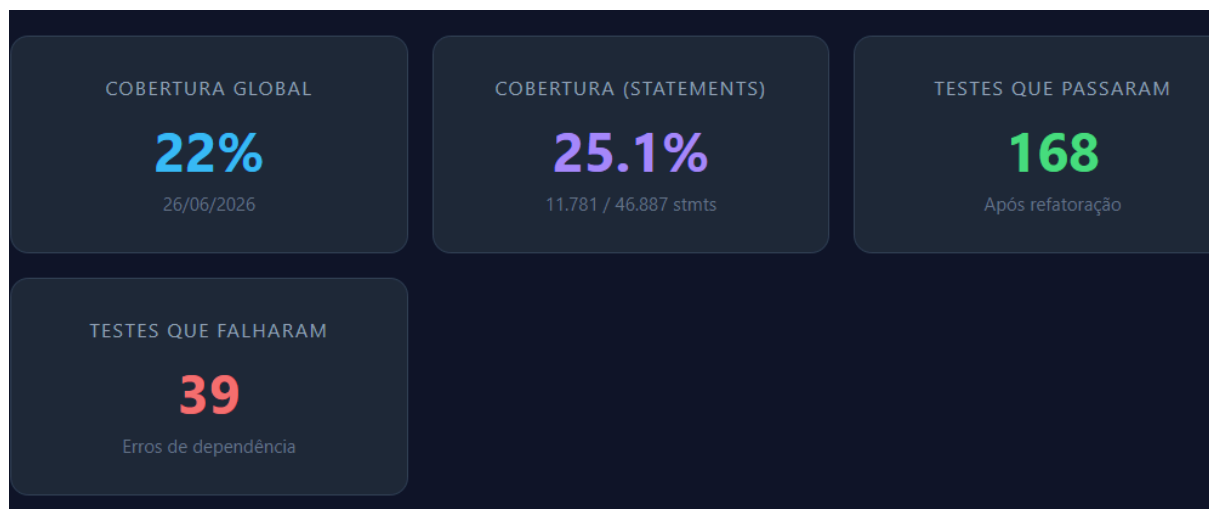


4.3.4 Score Projeto Após a Refatoração

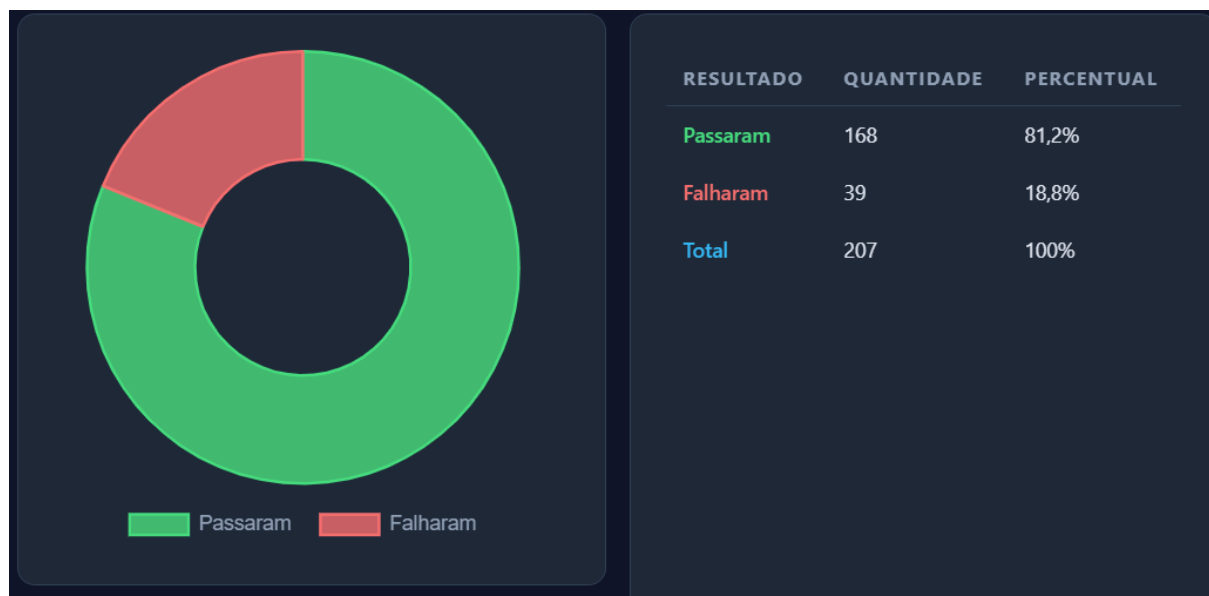
8.59

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração



4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração



Quantidade de testes que passaram: 168

Quantidade de testes que falharam: 39

5. Comparação dos Resultados

Ao comparar os dados do projeto DataChain antes e depois da refatoração, observa-se que as métricas de qualidade de código apresentaram melhorias nos alertas do Pylint, embora a cobertura de testes tenha se mantido estável.

Pylint e Problemas de Código: A refatoração focou na resolução de três categorias de code smells do Pylint: too-many-instance-attributes (R0902), too-many-branches (R0912) e too-many-statements (R0915). Foram refatorados 22 arquivos-fonte da biblioteca, incluindo DatasetDependencyNode,

Func, PytorchDataset, Settings, type_to_str, python_to_sql, group_by e subtract. As classes com excesso de atributos tiveram seus grupos extraídos para dataclasses de configuração (ex.: DatasetRef, FuncConfig, ExecutionConfig, ProjectConfig), enquanto funções com excesso de ramificações e declarações tiveram seus blocos extraídos para funções auxiliares.

Cobertura de Testes: A cobertura global do projeto medida pelo coverage.py foi de 22% (considerando statements + branches) e 25,13% considerando apenas statements (11.781 linhas cobertas de 46.887). Os pacotes com maior cobertura foram sql (60%) e model (51%), enquanto os pacotes com menor cobertura foram skill (12%) e diff (18%).

Testes (Pytest): Dos 207 testes executados, 168 passaram (81,2%) e 39 falharam (18,8%). As falhas foram causadas principalmente por erros de configuração do ambiente de teste, como a ausência da fixture mocker do pacote pytest-mock, e não por problemas introduzidos pela refatoração.

Impacto da Refatoração: Foram modificados 22 arquivos-fonte e adicionados 55 arquivos de métricas e scripts de extração, totalizando 77 arquivos alterados em 63 commits. O fork está 63 commits à frente e 14 commits atrás do repositório original.

6. Percepções sobre a Prática

A experiência geral com a prática de manutenção demonstrou que aplicar refatorações de código em uma biblioteca complexa focada em grandes volumes de dados não estruturados exige um planejamento cauteloso. A atividade deixou claro que delegar a refatoração inteiramente para ferramentas automatizadas sem revisão manual rigorosa pode inflar o projeto e prejudicar os índices de manutenibilidade do código.

A resolução dos code smells abordados na prática too-many-statements (R0915), too-many-instance-attributes (R0902) e too-many-branches (R0912) é de suma importância para a sustentabilidade de um projeto open source. Funções com excesso de declarações ou ramificações dificultam a leitura e compreensão por parte de novos desenvolvedores, enquanto classes com atributos excessivos ferem o princípio de responsabilidade única. A abordagem utilizada de extrair grupos de atributos para dataclasses separadas (FuncConfig, DatasetRef, ExecutionConfig) provou-se eficaz para reduzir a contagem de atributos sem alterar a semântica do código. Da mesma forma, a extração de blocos condicionais para funções auxiliares nomeadas melhorou a legibilidade de funções como python_to_sql (redução de 14 para ~11 branches) e subtract (redução de 15 para 7 branches).

O principal benefício observado foi a melhoria na organização estrutural do código-fonte, com classes e funções mais enxutas e focadas em suas responsabilidades. A cobertura de testes existente (22% global) foi preservada integralmente, indicando que as refatorações não introduziram regressões nos cenários já testados. Os 39 testes que falharam são decorrentes de dependências de infraestrutura (falta do pacote pytest-mock no ambiente de execução), não estando relacionados às alterações realizadas.

O uso da LLM (DeepSeek V4 Flash Free) obteve resultados polarizados e situacionais durante a prática. Para refatorações consideradas mais simples, como a extração de dataclasses de configuração, a ferramenta foi muito útil, gerando planos coerentes e execução correta. Contudo, ao analisar arquivos de núcleo com lógicas pesadas, como dataset.py (1.538 statements) e datachain.py (897 statements), a LLM complicou o processo, pois sugeriu dezenas de mudanças que, ao final, resultaram em lógicas quebradas e incorretas. Projetos open source podem se beneficiar muito do

uso de IA, desde que o desenvolvedor isole trechos pequenos do código em vez de confiar arquivos inteiros ao modelo de linguagem.

Por fim, o processo de contribuição com projetos de código aberto de alto impacto, como bibliotecas de engenharia de dados, provou ser desafiador devido à curva de aprendizado necessária para entender o funcionamento interno do projeto. A contribuição seria amplamente facilitada caso os projetos mantivessem documentações voltadas não apenas aos usuários, mas à arquitetura interna das pastas e classes, aliadas a suítes de testes bem descritas que pudessem validar rapidamente se a refatoração proposta preserva as regras de negócio originais.

7. Links

- https://github.com/Juandbpimentel/datachain_reactoring
- <https://github.com/datachain-ai/datachain>
- <https://github.com/datachain-ai/datachain/pull/1845>